

Relatório 28 - Projeto Final

Lucas Scheffer Hundsdorfer

1 - Introdução

Como proposta do projeto final, eu decidi criar um modelo classificatório de visão computacional, utilizando uma base de dados de fundo de olho com 4 classes, catarata, retinopatia diabética, glaucoma e normal. Meu objetivo com esse projeto é além de criar uma rede neural customizada que se adapte aos parâmetros das classes, e também utilizar de um modelo pré treinado e fazer comparações de quando é mais recomendado criar sua rede neural ou fazer o fine tuning em um modelo já pronto, trazendo além das métricas de validação do modelo, também trazendo uma interpretação dos resultados. E dentro desse relatório vai ser feito o acompanhamento, desde a base de dados escolhida como os modelos, métodos utilizados, testes feitos e também os resultados.

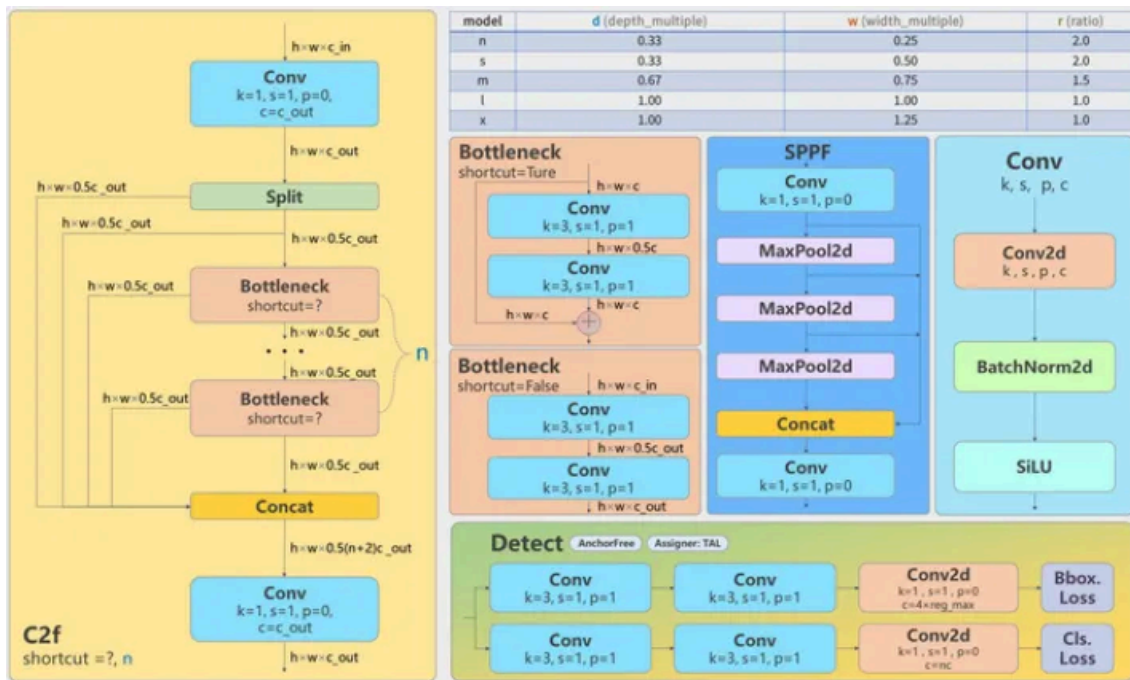
2 - Base de dados

A base de dados utilizada foi uma achada no kaggle:
<https://www.kaggle.com/datasets/gunavenkatdoddi/eye-diseases-classification>
dividida em 70% para treino, 15% para validação e os outros 15% para testes. A divisão foi feita através de um script em python que percorre cada diretório de classe embaralha-os e faz o particionamento proporcional, como o dataset disponibilizado já está balanceado o mesmo se manteve após a divisão.

3 - Modelos

O modelo pré-treinado que eu utilizei para fazer o fine tuning foi o yolo v8, ele é um modelo bem concreto e conhecido. O yolo é útil para vários tipos de tarefas além da classificação, de instância, pose/keypoints, detecção orientada. É oferecido vários tamanhos do modelo, Nano (n), Small (s), Medium (m), Large (l) e Extra-large (x).

Essa é arquitetura do YOLO v8:



Composta por 3 partes, Backbone, Neck e Head. Cada parte tem sua função dentro da arquitetura para poder ser feita toda a leitura e classificação da imagem.

Backbone é a primeira parte da rede neural, é onde a imagem entra e lá é extraído todas as características dela, transformando os pixels em valores numéricos. Utiliza-se de uma arquitetura baseada em CSPDarknet, aprimorada com blocos C2f que aumentam a eficiência e reduzem o custo computacional.

A segunda parte é o Neck, essa parte da estrutura age como um intermediário entre o backbone e o head, onde as informações são agregadas e combinadas, garantindo que objetos pequenos e grandes possam ser detectados com a mesma eficácia. Essa parte é baseada em PAnet (Path Aggregation Network).

A última parte o Head é o responsável pela saída da rede neural, ou seja as previsões finais, para cada tipo de modelo, classificação, detecção entre outros. Essa head é uma anchor-free, o que é uma novidade entre os modelos da Ultralytics e traz uma melhora referente a diferentes proporções e orientações dos objetos. Essa é toda a base da arquitetura do YOLO v8 e é por causa dessa arquitetura que esse modelo se demonstra tão eficiente nas suas tarefas.

Já o modelo que eu criei tem está arquitetura:

```
model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Rescaling(1./255),

    #primeiro bloco convolucional
    layers.Conv2D(64, (3, 3), padding='same', activation='relu', input_shape=(224,224,3)),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.SpatialDropout2D(dropout),

    #segundo bloco convolucional
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.SpatialDropout2D(dropout),

    #terceiro bloco convolucional
    layers.Conv2D(256, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(256, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.SpatialDropout2D(dropout),

    #quarto bloco convolucional
    layers.Conv2D(256, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(256, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.SpatialDropout2D(dropout),

    #camada densa
    layers.GlobalAveragePooling2D(),
    layers.Dense(512, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.4),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(dropout),
    layers.Dense(4, activation='softmax')
])
```

Também é uma rede neural convolucional, e é composta por 4 blocos de convolução seguidos e uma última camada densa, seguindo uma lógica de extração progressiva de características.

Dentro dos blocos de convolução temos camadas de convolução, batch normalization, max pooling e o dropout, as camadas convolucionais ficam responsáveis por identificar os padrões e aplicar filtros que geram os mapas de ativação correspondentes às características, todas as camadas estão utilizando a função de ativação ReLu que introduz uma não linearidade e acelera o treinamento. Já os batch normalizations servem para estabilizar o processo todo, ajustando a média e a variância das saídas anteriores, por isso sempre tem uma após a convolução. As camadas de max pooling servem para reduzir a dimensionalidade dos mapas de ativação que a camada convolucional geral, diminuindo o custo computacional. E os spatial dropouts são dropouts especializados para redes neurais convolucionais, pois invés de desativar apenas neurônios eles desativam mapas inteiros de ativação, evitando um possível overfitting. E a arquitetura demonstra que a cada bloco convolucional o número de filtros gerados cresce de 64 até 256, isso foi feito de maneira proposital para as primeiras camadas obterem as características mais simples e as mais profundas camadas obterem as características mais complexas e abstratas.

A camada densa vem após toda a extração das características das imagens, e é onde se faz a classificação, onde cada imagem é associada a uma probabilidade de cada classe, esse bloco também tem dropout e batch normalization pelos mesmos motivos.

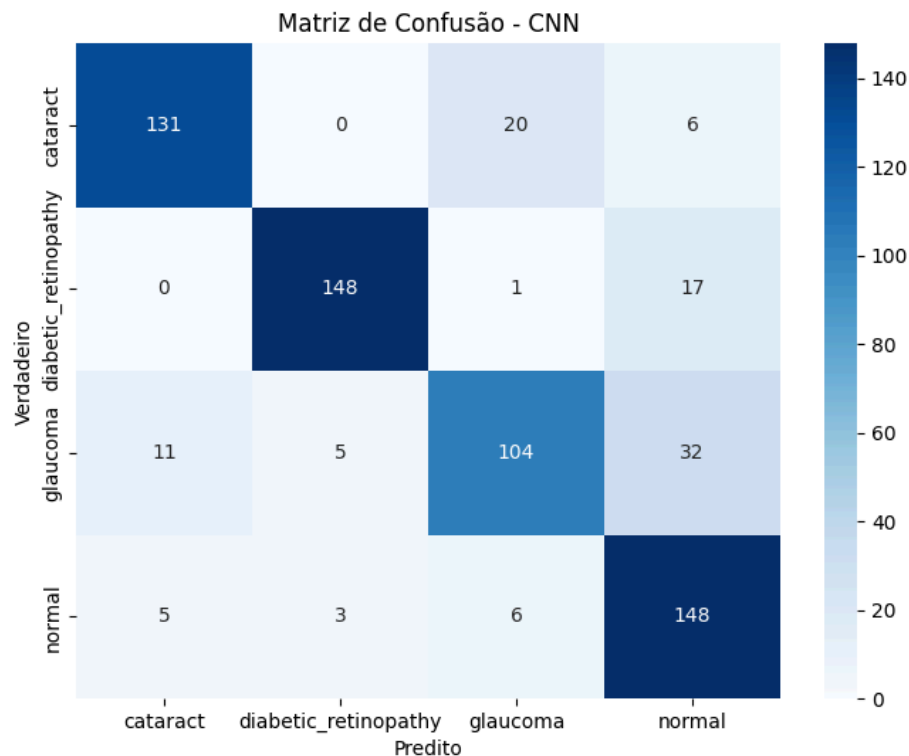
4 - Métodos utilizados

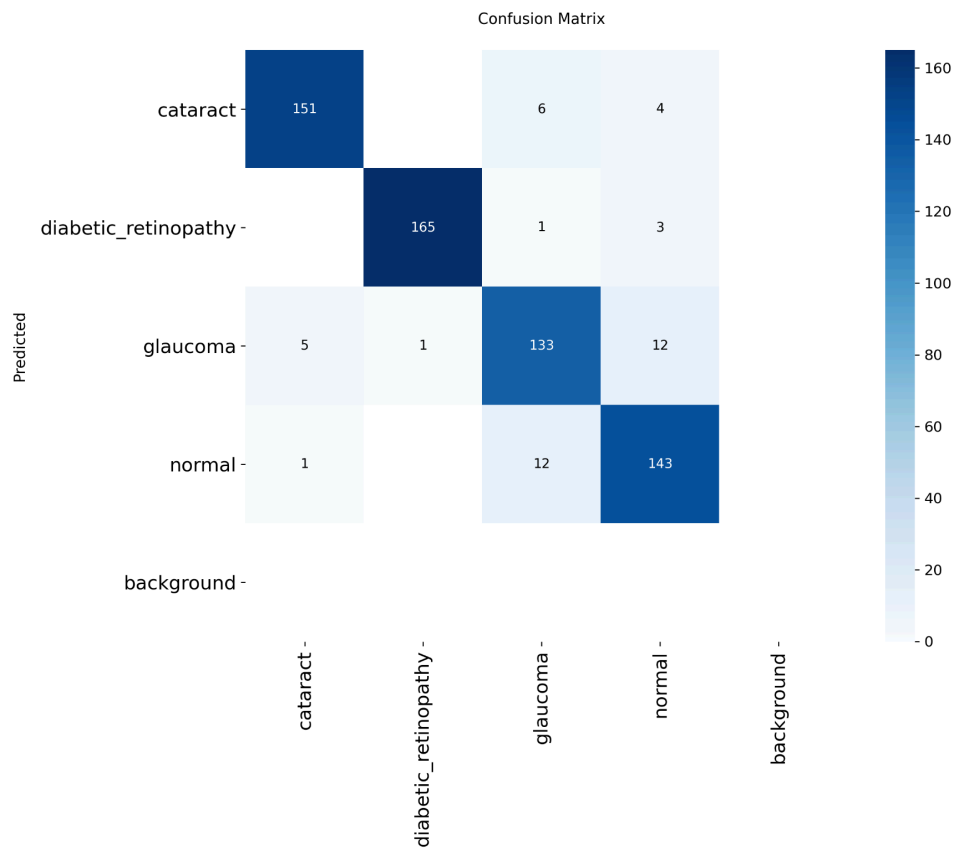
Métodos utilizados dentro do modelo criado, o pré-processamento dos dados foi a normalização dos pixels para ficar entre 0 e 1. Foi utilizado um grid search para podermos achar os melhores parâmetros para a rede, foram testadas 27 combinações diferentes de learning rate, dropout e weight decay sendo que a melhor combinação foi: {'lr': 0.0002, 'weight_decay': 0.0001, 'dropout': 0.15}. Foi definido o tamanho da imagem como 224x224 pixels, e o batch size de 32. Foi utilizado do data augmentation tanto para reduzir as chances de overfitting e tanto para generalização do modelo, foi aplicadas transformações leves por se tratar de um dataset clínico e não era o ideal distorções muito grandes, também é ideal ter o mínimo de data augmentation pois imagens clínicas não são tão fáceis de se obter e é feito esse processo para gerar um aumento na quantidade de dados. E por fim o treinamento foi feito com uma quantidade total de 100 épocas, utilizando o otimizador AdamW com agendamento do learning rate para não ficar preso e também utilizando o Early Stopping para evitar treinos demais sem a melhora do modelo. Após o primeiro treinamento foi feito um segundo treinamento em cima do melhor modelo, houve uma melhoria nas métricas porém nada muito alarmante, e isso se demonstra muito provavelmente pelo fato que o modelo já alcançou o ponto de saturação de aprendizado em relação a esse conjunto de dados, as características de cada classe foram suficientemente aprendidas, por isso mais treinos com a mesma base de dados não trará ganhos mensuráveis.

Primeiramente o modelo escolhido foi yolov8n-cls, versão nano do modelo com foco em classificação. De pré-processamento foi feito o redimensionamento para 224x224 que nem no modelo próprio, as imagens foram convertidas para RGB e os pixels normalizados para valores entre 0 e 1, e um batch size de 16. Foi aplicado o data augmentation também de uma forma leve apenas para evitar o overfitting e ganhar uma generalização. Foram executadas 100 épocas, foi utilizado o AMP(automatic mixed precision) que combina operações entre float16 e float32 economizando tempo e custo computacional, não foi utilizado dropout nesse modelo, e para controlar a subida e descida do learning rate o warmup epochs foi setado como 3.

5 - Resultados

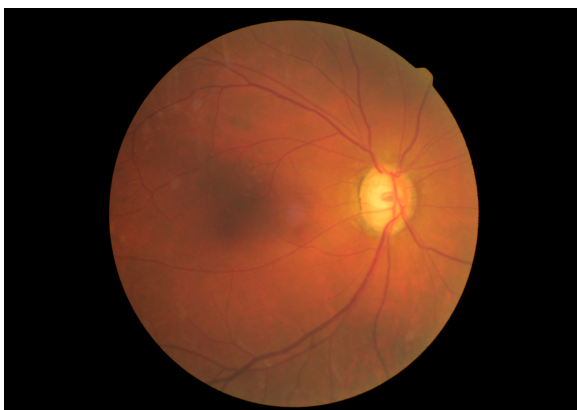
Após os treinamentos chegamos nessas duas matrizes de confusão



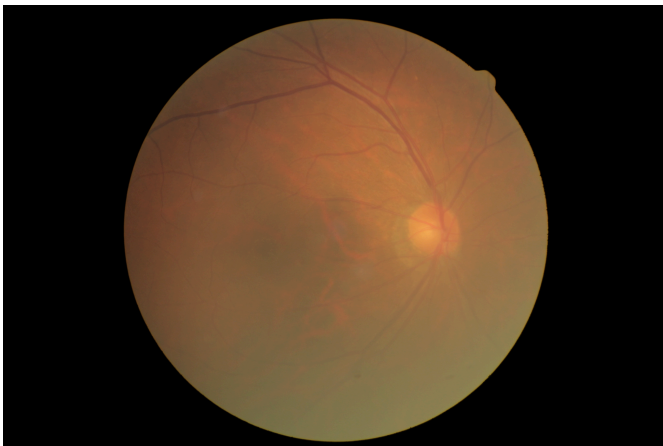


A primeira matriz do modelo criado e a segunda do modelo pré-treinado. Claramente o YOLO se saiu melhor, agora só vou detalhar os possíveis motivos e avaliar os resultados de ambos.

Essa diferença das matrizes de confusão se dá devido ao fato que o YOLO conseguiu atingir 93,2% de acurácia dentro do dataset de validação e 92,9% dentro do dataset de testes, já as métricas de precisão ficou em 0.9282, de recall em 0.9293 e o f1-score em 0.9287 se demonstrando um modelo confiável. E o erro mais comum foi entre glaucoma e normal, que são realmente mais parecidos e com poucas diferenças visuais:



Sendo a primeira imagem a de glaucoma e a segunda normal. Já a catarata e retinopatia diabética que são mais perceptíveis:



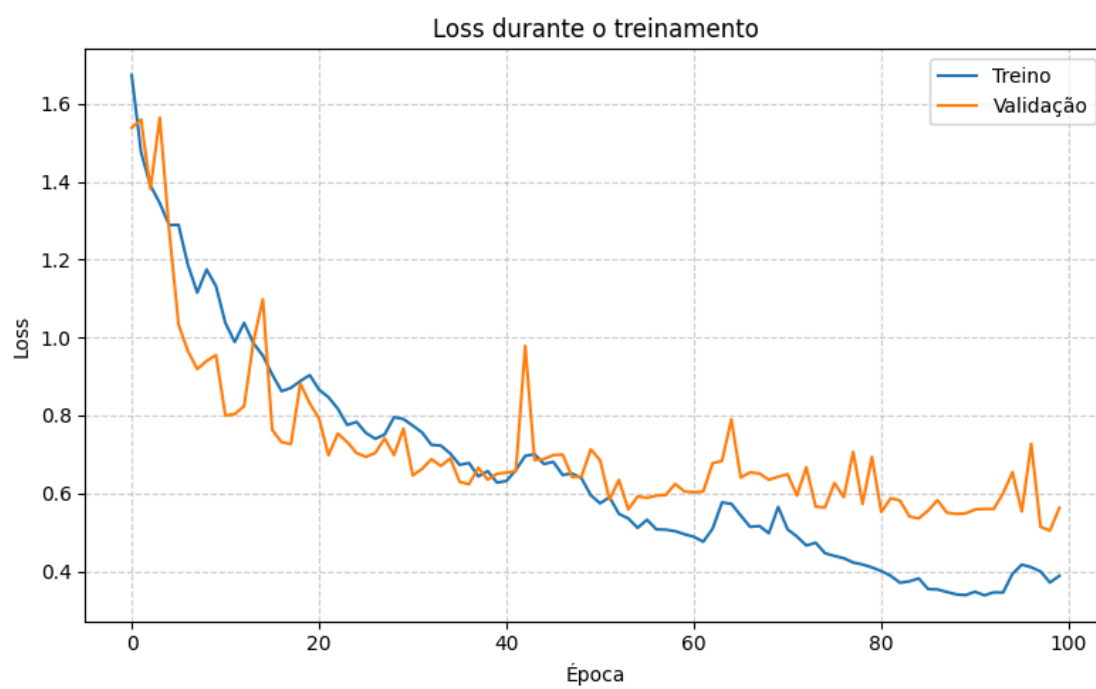
são as que têm menos erros.

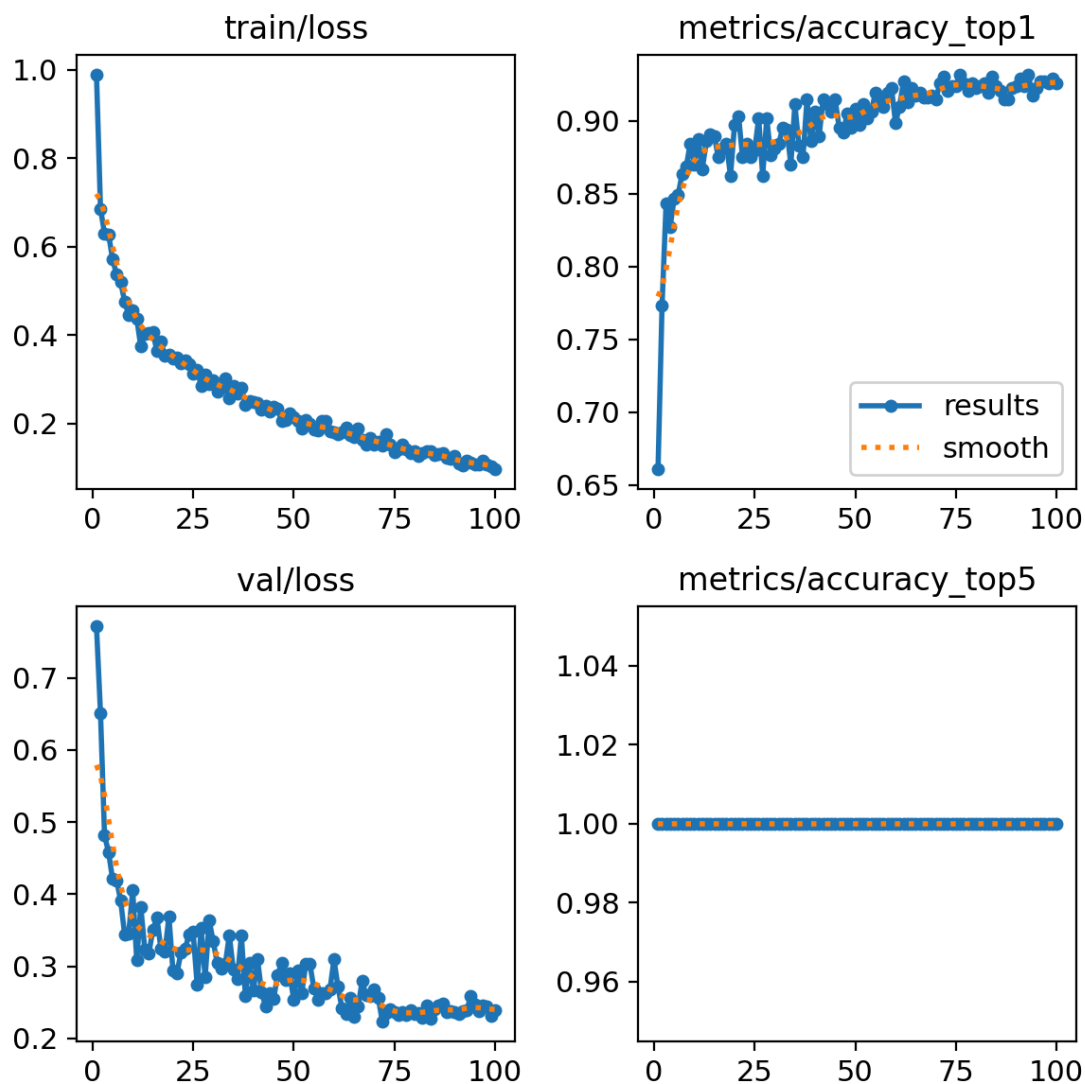
Agora o modelo criado, trouxe uma acurácia total um pouco menor de 83% nos dados de teste e nos dados de validação 82% de acurácia, com alguns erros a mais para classificar como glaucoma, pois não é tão evidente, e com uma certa facilidade a mais com as doenças fáceis de serem classificadas. Outras métricas importantes, o recall que é quanto que o modelo conseguiu prever corretamente, ficou em 0,83. A precisão que marca quantos das imagens que ele classificou como positivas ele acertou de fato, ficou em 0,84. E o f1-score é a média entre essas duas estatísticas e ficou em 0,83. Esses dados são importantes porque como se trata de um dataset clínico a precisão não é tão importante como o recall, já que é melhor alertar falsamente (um falso positivo) do que deixar passar um caso real (um falso negativo). O modelo customizado trouxe resultados sólidos, com algumas melhoras poderia se tornar melhor que o modelo customizado mas provavelmente iria trazer um custo computacional bem maior. Após o fine tuning do melhor modelo as métricas se atualizaram para 0,82 de f1-score e recall e a precisão em 0,83:

	precision	recall	f1-score	support
cataract	0.90	0.83	0.86	157
diabetic_retinopathy	0.84	0.95	0.89	166
glaucoma	0.80	0.72	0.75	152
normal	0.78	0.79	0.78	162
accuracy			0.83	637
macro avg	0.83	0.82	0.82	637
weighted avg	0.83	0.83	0.82	637

Agora os valores de loss tanto de validação e de treino durante as épocas dos treinos:

Primeiro a CNN:



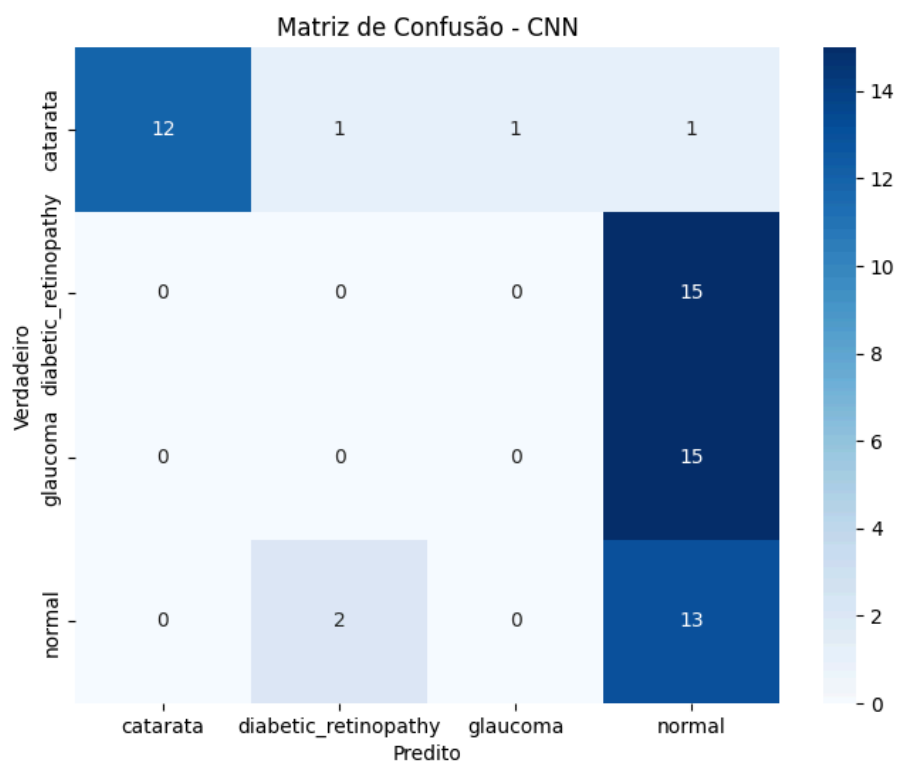


A curva de perda demonstra uma convergência estável entre treino e validação, indicando que o modelo customizado foi capaz de aprender padrões relevantes sem sinais expressivos de sobreajuste. A diferença entre as curvas é pequena, sugerindo boa generalização dentro do conjunto de dados utilizado.

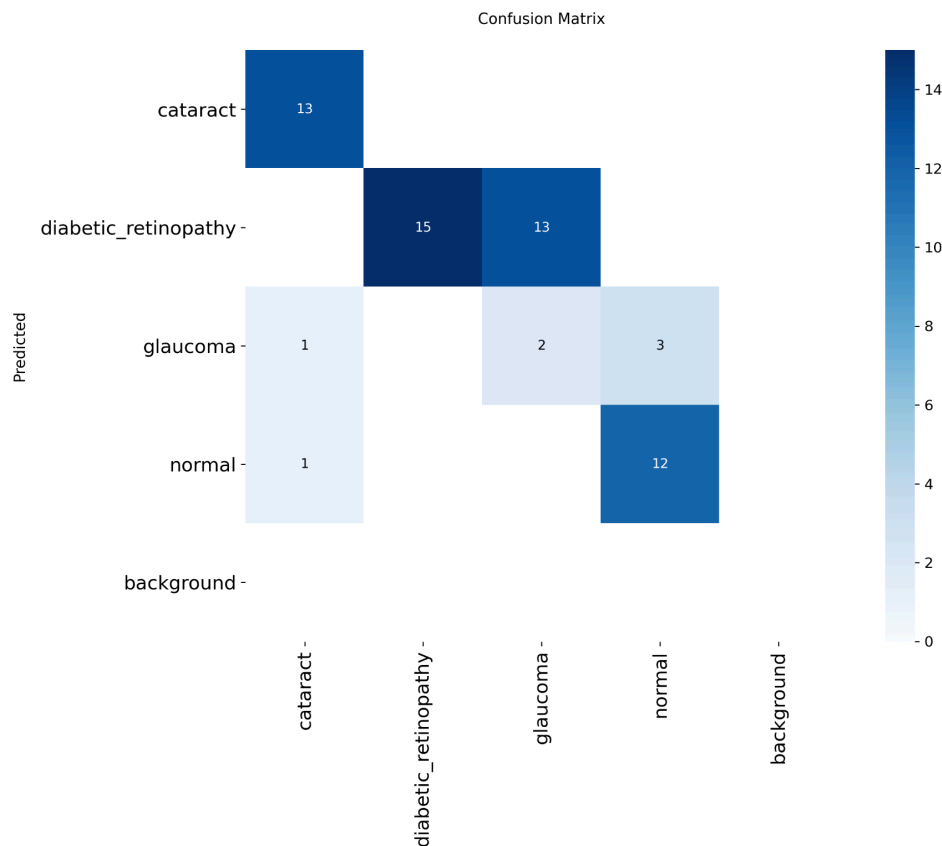
6 - Testes em dataset externo

Após os resultados obtidos com o conjunto de teste, foi realizada uma validação adicional utilizando um dataset externo que continha 15 imagens de cada classe. Entretanto, como as imagens clínicas desse conjunto apresentam condições de captura diferentes variações de iluminação, equipamentos, resolução e até protocolo de exame, o modelo apresentou redução de desempenho. Esse comportamento é esperado em aplicações médicas, uma vez que a padronização das imagens é crucial para a generalização do modelo.

A CNN trouxe resultados bem fracos aplicados a esse dataset, com uma acurácia de 0,4167, o que é bem ineficaz. Porém tem um motivo claro para esse comportamento, o modelo foi treinado a partir do zero, com uma arquitetura relativamente rasa e dependente das características específicas do dataset original. Isso significa que ele “aprendeu” apenas as condições controladas das imagens iniciais, iluminação mais uniforme, enquadramento consistente e qualidade visual padronizada. Quando exposto a um domínio diferente, como o dataset clínico externo, cheio de variações mais agressivas de foco, brilho, ruído e posicionamento, a CNN simplesmente não possuía repertório suficiente para reconhecer esses padrões novos. Aqui está a matriz de confusão deste dataset:



Essa limitação contrasta diretamente com o YOLO, que obteve desempenho muito superior de 0,6999 de acurácia, porque utiliza um backbone profundo pré-treinado em milhões de imagens e um pipeline interno de augmentations mais diversificado. Essa diferença é fundamental para entender por que a generalização da CNN foi tão limitada no dataset externo. Aqui está a matriz de confusão:



7 - Conclusões

Minhas conclusões com este trabalho permitiu compreender um pouco mais sobre a diferença entre modelos pré-treinado e modelos desenvolvidos do zero, mostrando que ambas abordagens têm papéis complementares e que são dependentes do objetivo e recursos do projeto.

Para uma análise apenas de resultados nesse caso, o melhor modelo foi claramente o YOLO, trazendo uma acurácia de 91% no dataset de treino e quase 70% no dataset externo, superior a CNN customizada, porém esses resultados só refletem o que acontece nesse contexto específico com essa quantidade de dados. O YOLO se demonstrou superior também no tempo de treinamento bem inferior a CNN, demorando apenas 9 minutos e 14 segundos e a CNN para completar 100 épocas demorou 1 hora 29 minutos e 6 segundos.

Modelos pré-treinados são recomendados para uso quando você precisa economizar tempo, dados e recursos computacionais, pois eles já foram treinados em grandes conjuntos de dados e aprendem padrões gerais que podem ser adaptados para novas tarefas por fine-tuning. Eles são indicados especialmente quando você tem um conjunto de dados pequeno ou limitado. Modelos customizados, por outro lado, são recomendados quando o objetivo é máxima personalização e acurácia para um domínio específico. Eles podem ser justificados se o seu problema requer aprendizado de padrões muito

particulares ou conteúdos que não estavam presentes no treinamento original do modelo pré-treinado. No entanto, treinamentos customizados demandam mais dados, tempo, recursos computacionais e expertise para ajustar hiperparâmetros e evitar overfitting.

A comparação entre modelo customizado e pré-treinado mostra que a escolha da abordagem não deve ser guiada apenas pelo desempenho final, mas pelo equilíbrio entre generalização, custo computacional, tamanho do dataset e objetivo do projeto. Em contextos com poucos dados e alta variabilidade, modelos pré-treinados oferecem vantagens significativas; já modelos customizados se tornam mais valiosos quando se busca controle total sobre a arquitetura e estudo científico aprofundado